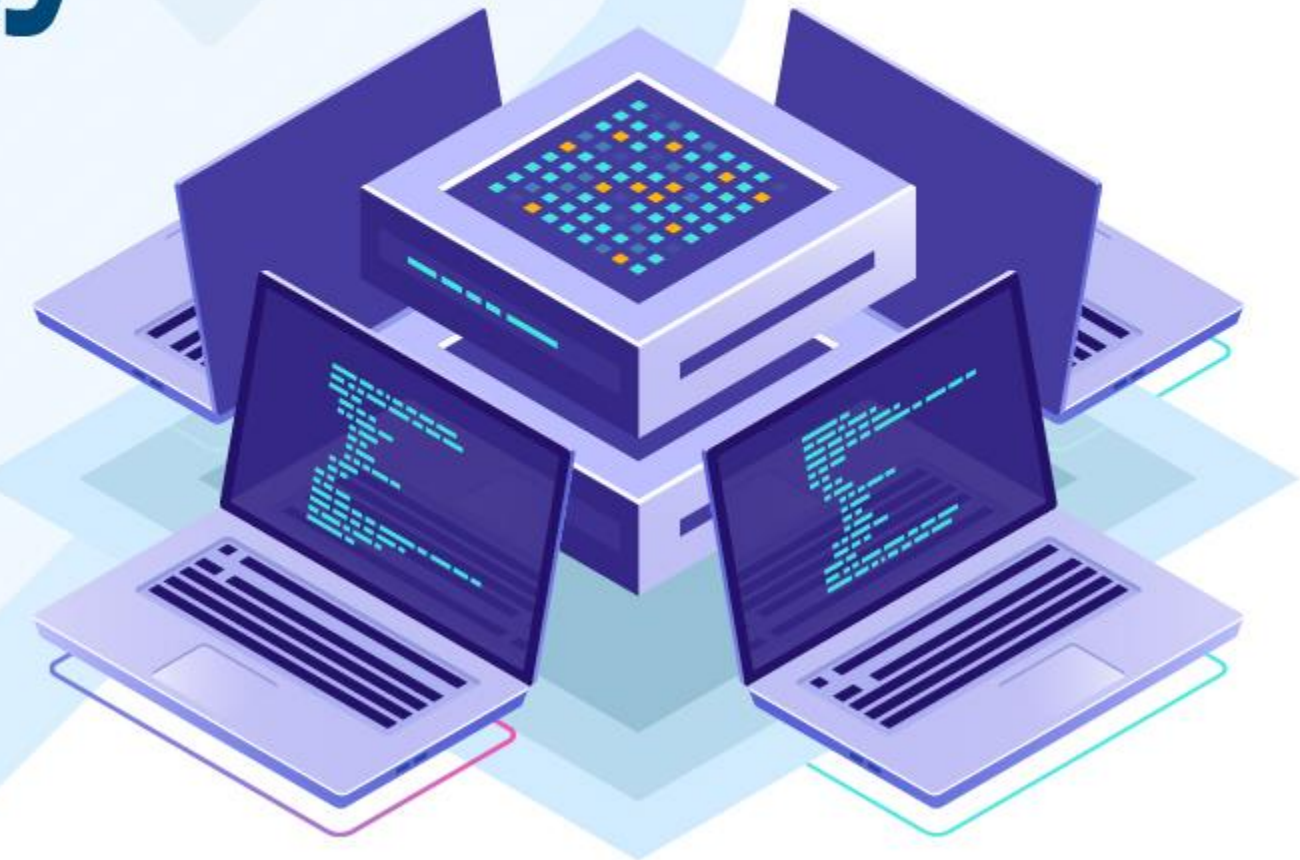
# Information Technology

Year: 2026-2027

Fall Semester

- Midpoint Circle Drawing Algorithm

- **Dr. Yasmine Mahmoud**



EELU

الجامعة المصرية للتعلم الإلكتروني الأهلية

THE EGYPTIAN E-LEARNING UNIVERSITY

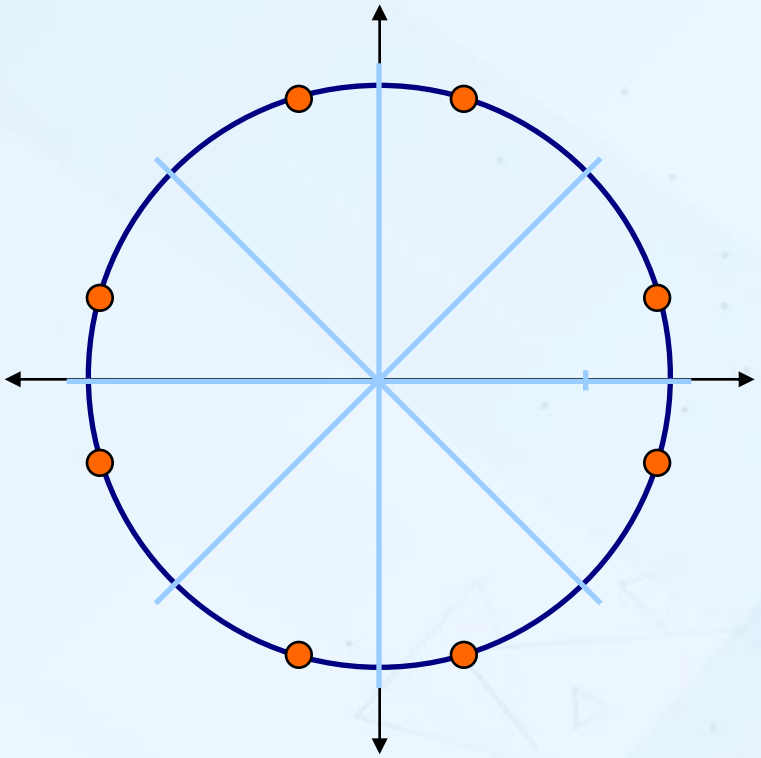
# Midpoint Circle Drawing Algorithm



EELU

# CIRCLE

- The set of points that are all at a given distance ‘r’ from a center position (Xc,Yc).



# A Simple Circle Drawing Algorithm

Scan Converting Circles

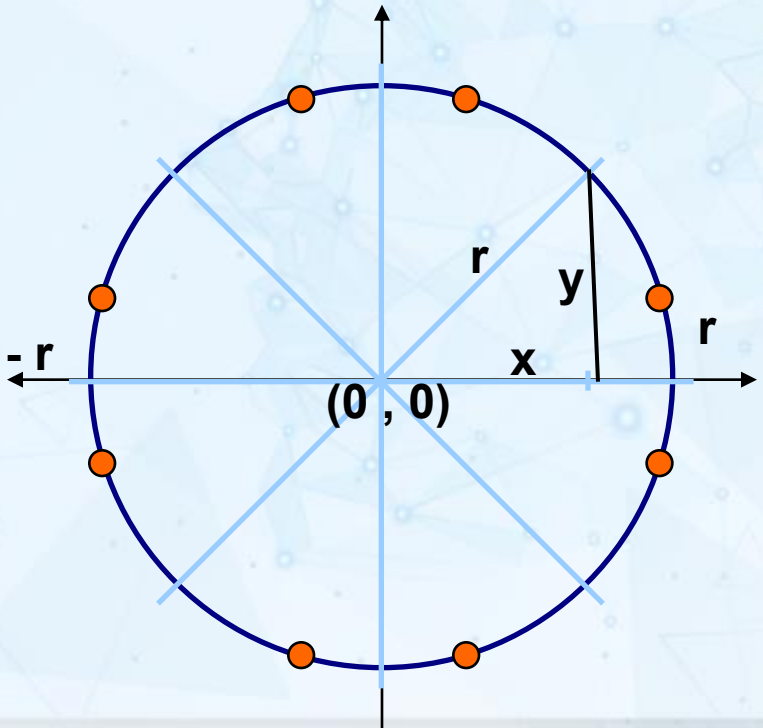
- The equation for a circle is:

$$x^2 + y^2 = r^2$$

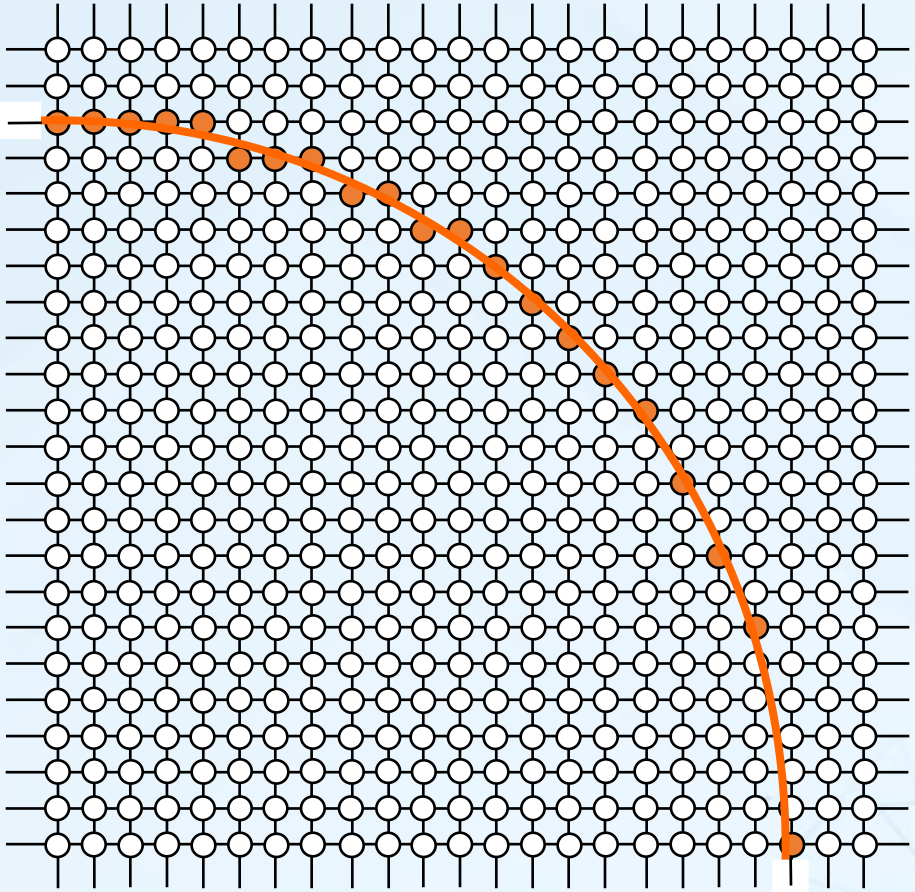
- where r is the radius of the circle

- So, we can write a simple circle drawing algorithm by solving the equation for $y$ at unit $x$ intervals using:

$$y = \pm\sqrt{r^2 - x^2}$$



# A Simple Circle Drawing Algorithm (cont…)



$$y = \pm\sqrt{r^2 - x^2}$$

Ex:
r = 20

$$y_0 = \sqrt{20^2 - 0^2} \approx 20$$

$$y_1 = \sqrt{20^2 - 1^2} \approx 20$$

$$y_2 = \sqrt{20^2 - 2^2} \approx 20$$

$$\vdots$$

$$y_{19} = \sqrt{20^2 - 19^2} \approx 6$$

$$y_{20} = \sqrt{20^2 - 20^2} \approx 0$$

EELU

# A Simple Circle Drawing Algorithm (cont…)

- However, unsurprisingly this is not a brilliant solution!
  - Firstly, the resulting circle has large gaps where the slope approaches the vertical
  - Secondly, the calculations are not very efficient
    - The square (multiply) operations
    - The square root operation – try really hard to avoid these!
- We need a more efficient, more accurate solution

# Scan Converting Circles

- Parametric:

$$x = R\cos\theta$$

$$y = R\sin\theta$$

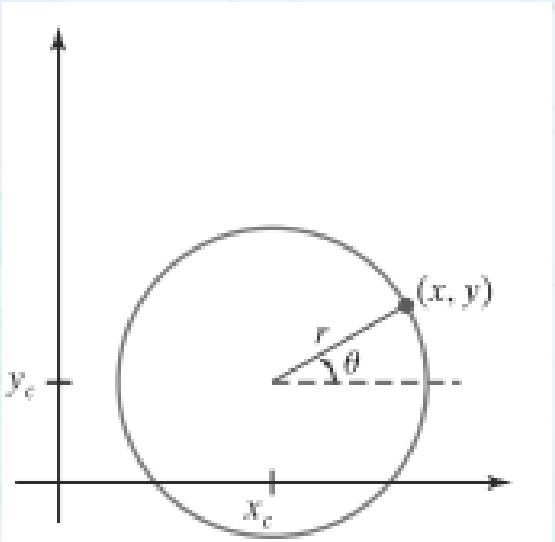



**FIGURE 11**
Circle with center coordinates $(x_c, y_c)$ and radius $r$.

Draw quarter circle by stepping through the angle from 0 to 90

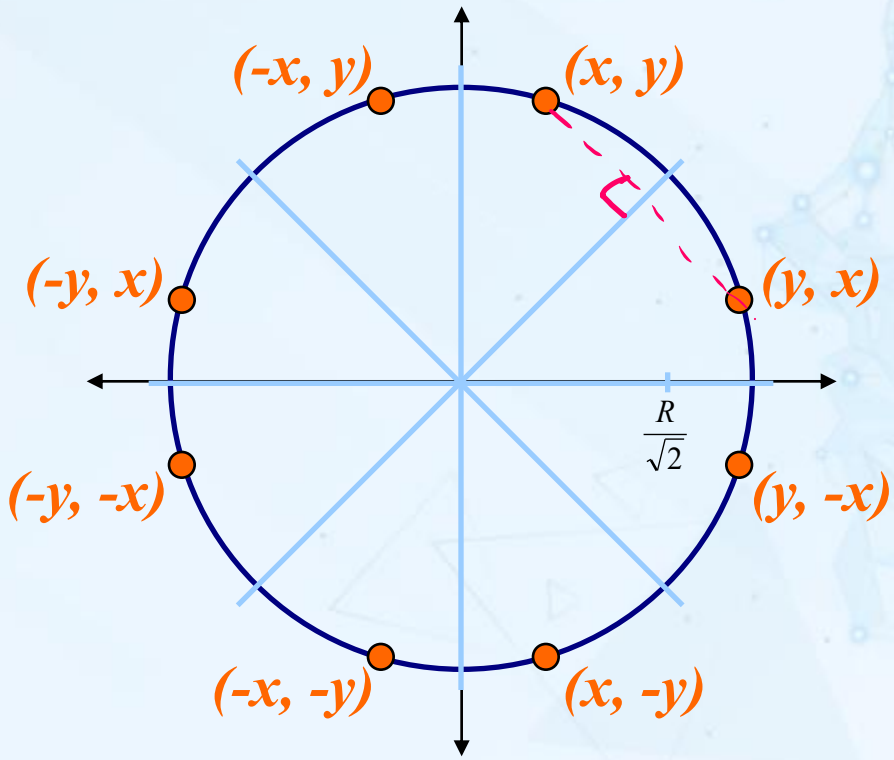
-avoids large gaps but still unsatisfactory

-How to set angular increment

Computationally expensive trigonometric calculations

# Eight-Way Symmetry

- The first thing we can notice to make our circle drawing algorithm more efficient is that circles centred at (*0, 0*) have *eight-way symmetry*



# Mid-Point Circle Algorithm

- Similarly to the case with lines, there is an incremental algorithm for drawing circles – the ***mid-point circle algorithm***
- In the mid-point circle algorithm we use eight-way symmetry so only ever calculate the points for the top right eighth of a circle, and then use symmetry to get the rest of the points.



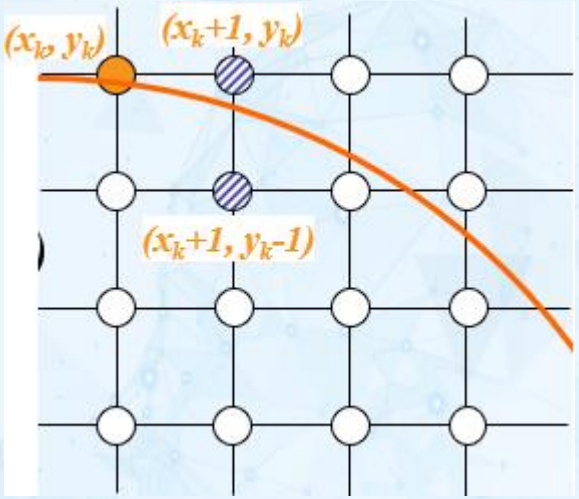
The mid-point circle algorithm was developed by Jack Bresenham, who we heard about earlier.



EELU

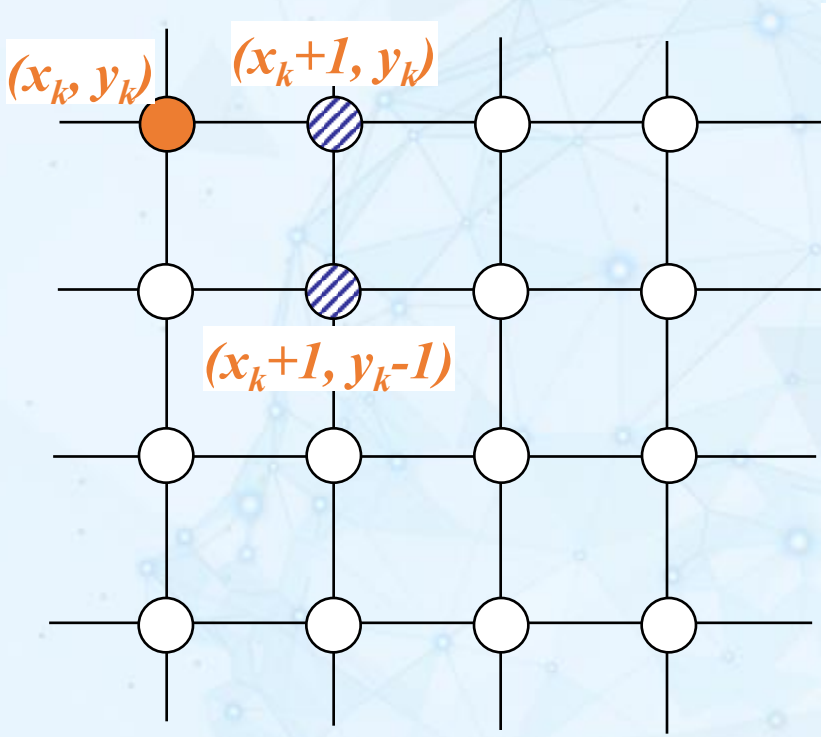
# Mid-Point Circle Algorithm (cont…)

- Assume that we have just plotted point $(x_k, y_k)$
- The next point is a choice between $(x_k+1, y_k)$ and $(x_k+1, y_k-1)$
- We would like to choose the point that is nearest to the actual circle
- So how do we make this choice?



# Mid-Point Circle Algorithm (cont…)

- Assume that we have just plotted point $(x_k, y_k)$
- The next point is a choice between $(x_k+1, y_k)$ and $(x_k+1, y_k-1)$
- We would like to choose the point that is nearest to the actual circle
- So how do we make this choice?



# Mid-Point Circle Algorithm (cont…)

- Let's re-jig the equation of the circle slightly to give us:
- $$f_{circ}(x,y) = x^2 + y^2 - r^2 \quad \ldots(1)$$ → $r^2 = x^2 + y^2$
- The equation evaluates as follows:

$$f_{circ}(x,y) \begin{cases} < 0, \text{ if } (x,y) \text{ is inside the circle boundary} \\ = 0, \text{ if } (x,y) \text{ is on the circle boundary} \\ > 0, \text{ if } (x,y) \text{ is outside the circle boundary} \end{cases}$$

- By evaluating this function at the midpoint between the candidate pixels we can make our decision



EELU

# Mid-Point Circle Algorithm (cont…)

- Assuming we have just plotted the pixel at $(x_k, y_k)$ so we need to choose between $(x_k+1, y_k)$ and $(x_k+1, y_k-1)$
- Our decision variable can be defined as: mid point b/w 2 points $(x_k+1, Y_k)$ and $(Xk+1, Y_k-1)$ is $[x_k+1, y_k-1/2]$
- $$p_k = f_{circ}(x_k + 1, y_k - \tfrac{1}{2})$$
$$= (x_k + 1)^2 + (y_k - \tfrac{1}{2})^2 - r^2 \quad \ldots 2$$

$$f_{circ}(x, y) = x^2 + y^2 - r^2$$

- If $\boldsymbol{p_k < 0}$ the midpoint is inside the circle and the pixel at $y_k$ is closer to the circle
- **Otherwise** the midpoint is outside and $y_k$-1 is closer



EELU

# Mid-Point Circle Algorithm (cont…)

- To ensure things are as efficient as possible we can do all of our calculations incrementally
- First consider: ( since $X_k+1 = X_{k+1}$)

$$f_{circ}(x, y) = x^2 + y^2 - r^2$$

$$p_{k+1} = f_{circ}\left(x_{k+1} + 1, y_{k+1} - \tfrac{1}{2}\right)$$

- or:

$$= [(x_k + 1) + 1]^2 + \left(y_{k+1} - \tfrac{1}{2}\right)^2 - r^2$$

$$p_{k+1} = p_k + 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1$$

- where $y_{k+1}$ is either $y_k$ or $y_k - 1$ depending on the sign of $p_k$



- The initial value of Pk is given by the circle function at the position **(0,r)** as,

$$p_k = f_{circ}(x_k + 1, y_k - 1/2)$$

$$= (x_k + 1)^2 + (y_k - 1/2)^2 - r^2$$

$$f_{circ}(x, y) = x^2 + y^2 - r^2$$

- Substituting $k = 0, Xk = 0, Yk = r$ in the above function results in,



EELU

# Mid-Point Circle Algorithm (cont…)

- The **first decision variable** is given as:

$$p_0 = f_{circ}(1, r - \tfrac{1}{2})$$
$$= 1 + (r - \tfrac{1}{2})^2 - r^2$$
$$= \tfrac{5}{4} - r$$

- 
- if **r** is an integer, then **P0** can be rounded to **P0= 1 – r.**
- Then if $p_k < 0$ then the next decision variable is given as:

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

- If $p_k > 0$ then the decision variable is:

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_k + 1$$



EELU